

ASCON-KECCAK AEAD: A Novel Hardware Architecture for Cryptographic Processing

Ritika, B. Srinivasu, Kamal Raj and Anupam Chattopadhyay

Abstract—This paper presents the first hardware implementation of ASCON-KECCAK AEAD algorithm, which is designed by replacing the ASCON sponge with the KECCAK sponge. The proposed hardware design is validated on Kintex-7 and Artix-7 FPGA platforms, achieving a high throughput of 3.32 Gbps for ASCON-KECCAK and 20 Gbps for KECCAK-p[1600,24] permutation, while maintaining optimized resource utilization with 6523 LUTs for ASCON-KECCAK²⁵⁶, 7126 LUTs for ASCON-KECCAK⁵¹², and 2970 LUTs for KECCAK-p[1600,24] on Kintex-7. Additionally, ASIC benchmarking further demonstrates its feasibility for hardware-constrained environments. With the rising demand for secure cryptographic solutions, ASCON-KECCAK emerges as a highly secure and efficient algorithm, offering enhanced protection against key recovery and data integrity breaches in encryption and decryption processes.

Index Terms—ASCON, AEAD, KECCAK, FPGA, Security

I. INTRODUCTION

The evolution of smart cities is closely related to the integration of technologies such as the Internet of Things (IoT) [1], artificial intelligence (AI) [2], blockchain [3], and 5G communication networks [4]. These systems operate in interconnected environments, where continuous interaction between devices, particularly in IoT framework, exposes them to increased risks that involve data breaches and privacy violations [5]. As cyber attacks become more advanced, conventional cryptographic techniques are proving inadequate, rendering many legacy encryption methods insufficient. This has created a critical need for security mechanisms that combine robustness with performance.

ASCON [6] is a family of lightweight cryptographic algorithms that include variants designed for authenticated encryption with associated data (AEAD) [7] as well as hashing functions. The ASCON sponge has a size of 320 bits, providing a security strength of 128-bits for the AEAD algorithm. The ASCON AEAD algorithm can function with alternative sponge while still adhering to the operational framework. While ASCON provides lightweight and efficient encryption, and KECCAK the core of SHA-3 offers strong cryptographic resilience and flexibility, their integration in a unified hardware design remains underexplored. Addressing this gap, our work introduces a combined ASCON-KECCAK AEAD architecture

FPGA implementations, targeting enhanced throughput and efficient resource usage. This design aims to deliver a secure, scalable cryptographic solution suited for the performance and security demands of next-generation hardware systems.

KECCAK permutation [8] is a cryptographic operation mainly used in the KECCAK family of algorithms. A single round of a KECCAK permutation comprises a series of five transformations, collectively referred to as step mappings. The permutation operates on an array of values, consisting of b -bit elements, referred to as the state. KECCAK-p[1600,24], the core function behind the SHA-3 [8] and SHAKE [9] standard, has exhibited strong resistance to conventional cryptanalytic techniques, making it a crucial building block for modern cryptographic protocols. A compact FPGA-based SHA-3 implementation [10] showcasing KECCAK's superior speed and security over BLAKE [11], JH [12], and Skein [13] for data integrity tasks. A unified FPGA-based cryptographic core AESHA-3, to perform both AES [14] and SHA-3 operations as per NIST standards [15]. A recomputing with rotated operands (RERO)-based error detection scheme for SHA-3 was proposed in [16]. The proposed approach enhances data integrity and encryption efficiency, making it well-suited for secure transactions, confidential communications, and authentication protocols.

Various FPGA implementations of ASCON are reported in literature, such as error detection schemes for ASCON-128, enhancing S-Box security using LUT-based and logic gate-based techniques are introduced in [17]. An energy-efficient ASIC implementation of ASCON optimized for IoT applications is presented in [18]. The work presented in [19] focuses on the ASCON-KECCAK AEAD algorithm, providing a software-based approach to its cryptographic operations. This work presents an optimized FPGA-based implementation of ASCON-KECCAK, offering significant advantages over software implementations by achieving high throughput, efficient resource utilization, parallel processing capabilities, and robust security. The main contributions of this paper are summarized as follows:

- To the best of our knowledge, this paper presents a first-ever hardware implementation of a novel cryptographic algorithm that integrates the ASCON framework with the KECCAK-p[1600,24] permutation to enhance efficiency and security.
- Two configurations are implemented, namely, ASCON-KECCAK²⁵⁶ and ASCON-KECCAK⁵¹² to provide a classical security level of 256 and 512 bits, respectively. Furthermore, these two offer a quantum security level of 128 and 256 bits, respectively.

Ritika and B. Srinivasu are with the School of Computing and Electrical Engineering, Indian Institute of Technology Mandi - 175075, India.

Kamal Raj is with Temasek Laboratories@NTU, Nanyang Technological University, Singapore - 637553.

Anupam Chattopadhyay is with the College of Computing and Data Science, Nanyang Technological University, Singapore, 639798. Email for correspondence: s23055@students.iitmandi.ac.in, srinivasu@iitmandi.ac.in, kamalraj@ieee.org, anupam@ntu.edu.sg

- The KECCAK permutation is optimized by refining the Rho (ρ) transformation, leveraging an AND gate for modular arithmetic to improve computational efficiency.

To validate the proposed architecture, the FPGA implementation on the Kintex-7 and Artix-7 are presented. ASCON-KECCAK²⁵⁶ and ASCON-KECCAK⁵¹² achieved an operating frequency of 217 MHz, utilizing 6523 LUTs and 7126 LUTs, respectively, attaining a throughput of 3.32 Gbps. KECCAK-p[1600,24] implementation reaches a frequency of 313 MHz, delivering a throughput of 20 Gbps. These results demonstrate ASCON-KECCAK's efficiency as a high-throughput, resource-optimized cryptographic core suited for secure hardware-level applications.

The structure of the paper is as follows: Section II outlines the background on ASCON and the KECCAK permutation. Section III details the proposed hardware architecture for ASCON-KECCAK and KECCAK-p[1600,24]. Section IV presents the implementation results and analysis, while Section V concludes the paper.

II. BACKGROUND

ASCON-128 and ASCON-128a are cryptographic primitives designed for encryption, decryption, and authentication. ASCON-KECCAK follow the AEAD algorithm specification, where the sponge function is replaced with the KECCAK sponge function [8], utilizing suitable parameter sets while preserving the security assurances of ASCON. Table I lists the parameters for the ASCON and ASCON-KECCAK variants.

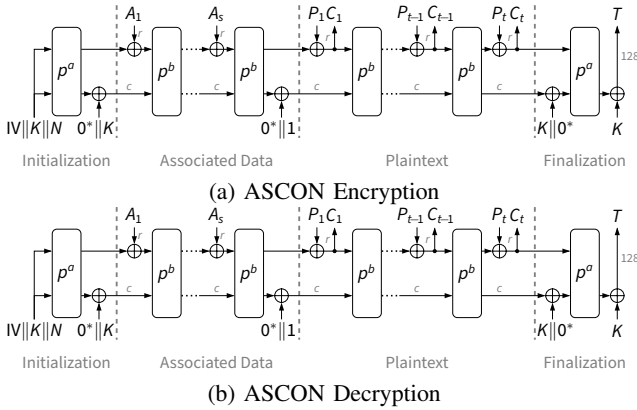


Fig. 1. ASCON Encryption and Decryption Architectures [6].

A. Encryption and Decryption with Associated Data

Fig. 1 illustrate the fundamental architecture of ASCON encryption and decryption, respectively. ASCON [6] family has a 320-bit state register, S . S is sectioned into rate (r) and capacity (c), defined as $c = 320 - r$. ASCON-KECCAK operates on a 1600-bit state register with rate (r), and capacity (c), specified as $c = 1600 - r$. The state register is initialized with Initialization Vector (IV), a 64-bit constant as well as key and $nonce$. At the initialization stage, p^a permutation is performed. Table II presents the initialization vectors (IV) used

TABLE I
PARAMETERS FOR ASCON AND ASCON-KECCAK VARIANTS [6]

Parameter	ASCON-128	ASCON-128a	ASCON-KECCAK ²⁵⁶	ASCON-KECCAK ⁵¹²
Key (K)	128	128	256	512
Nonce (N)	128	128	128 to 256	128 to 512
Rate (r)	64	128	1088	576
Capacity (c)	256	192	512	1024
Round (p^a)	12	12	24	24
Round (p^b)	6	8	24	24

in ASCON-KECCAK. The state S is updated as follows after initialization stage,

$$S \leftarrow p^a(S) \oplus (0^* || K)$$

The processing of associated data begins by partitioning it into r -bit blocks, with the final block padded using a 1 followed by 0s to ensure its length matches exactly r bits. Each block undergoes an xor operation with the r -bit internal state, followed by the p^b permutation. After this, a single bit of the internal state is undergoes an xor operation with 1. In the plaintext processing phase, each plaintext block is exored with the internal state to produce ciphertext, and then undergoes the p^b round permutation, except for the last block, which skips this step. Ciphertext processing follows a similar approach, extracting plaintext and updating the r -bit internal state using the ciphertext block, following the transformation:

$$S \leftarrow p^b(C_i || S_c), 1 \leq i \leq t$$

Finally, during the finalization step, a tag is generated during both encryption and decryption following a p^a round permutation. The tag, defined as follows, is used to verify processed data, with the chosen size satisfying $128 \leq t \leq key$.

$$T \leftarrow [S]^t \oplus [K]^t$$

TABLE II
INITIALIZATION VECTORS (IV) FOR ASCON-KECCAK VARIANTS

Variant	Tag Size	IV Value
ASCON-KECCAK ²⁵⁶	128-bit	2088181810000000
	256-bit	2088181820000000
ASCON-KECCAK ⁵¹²	128-bit	4048181810000000
	512-bit	4048181840000000

B. KECCAK Permutation

The KECCAK permutation follows the KECCAK-p[b, n_r] specifications, where b denotes the permutation width, and n_r represents the number of rounds as defined in [8]. The input and output states can be represented as 1600-bit strings or as $5 \times 5 \times 64$ arrays of bits for step mappings. Alternatively, if $A[x, y, z]$ represents the state as a $5 \times 5 \times 64$ array, its indices are integer triples (x, y, z) , where $0 \leq [x, y] < 5$ and $0 \leq z < 64$. This provides a three-dimensional representation of the state. The KECCAK-p[1600,24] permutation executes 24 rounds of transformation, incorporating five step mappings: $Theta$ (θ), Rho (ρ), Pi (π), Chi (χ) and $Iota$ (ι) represented in Fig. 2 [8]. The Algorithm 1 defines the operations executed in each layer of KECCAK-p[b, n_r] within a single round.

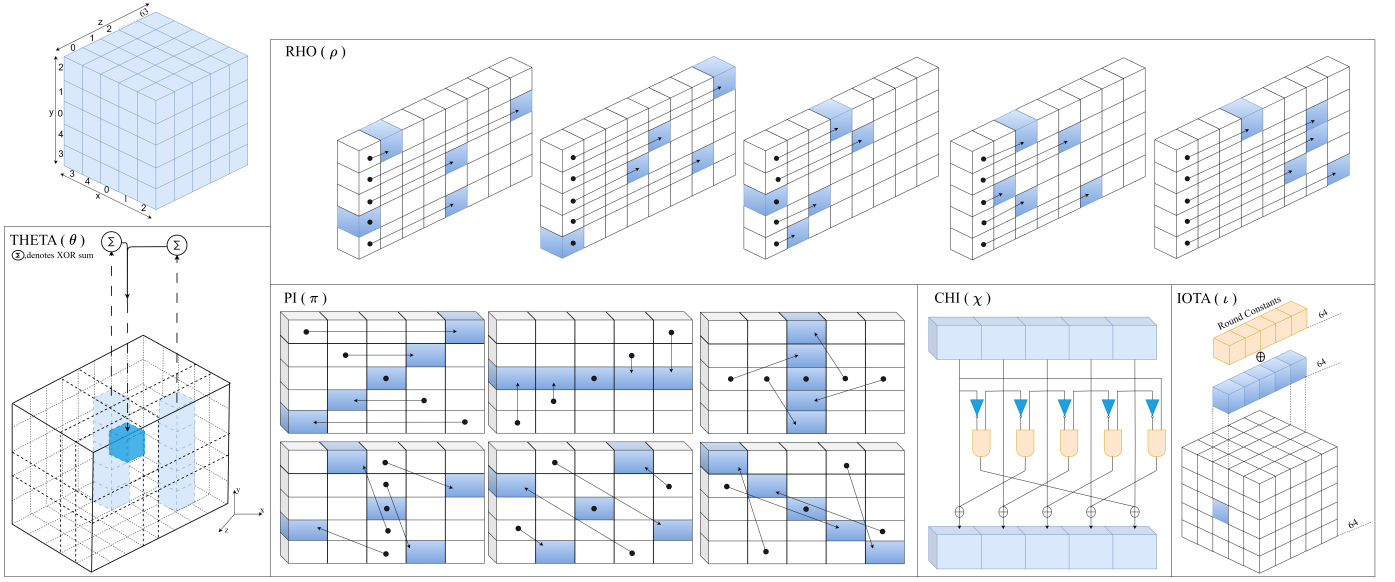


Fig. 2. Illustration of KECCAK Step Mappings: θ on a single bit, ρ for $b = 200$, π operating on a single slice, χ executed on a row and ι . [8]

Algorithm 1 KECCAK-p[b, n_r] Transformation

- 1: **Input:** State array $A[x, y, z]$
- 2: **Output:** Updated state array $A[x, y, z]$
- 3: **Theta1** (θ_1): $C[x, z] \leftarrow A[x, 0, z] \oplus A[x, 1, z] \oplus A[x, 2, z] \oplus A[x, 3, z] \oplus A[x, 4, z]$
- 4: **Theta2** (θ_2): $A[x, y, z] \leftarrow A[x, y, z] \oplus C[(x-1), z] \oplus C[(x+1), (z-1)]$
- 5: **Rho** (ρ): $A[x, y, z] \leftarrow A[x, y, (z - ((t+1)(t+2)/2))]$ for $0 \leq t < 24$, and:

$$\begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix}^t \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} x \\ y \end{pmatrix} \text{ in } \text{GF}(5)^{2 \times 2},$$

for $x = y = 0, t = -1$.

- 6: **Pi** (π): $A[x, y, z] \leftarrow A[y, (2x + 3y) \bmod 5, z]$
- 7: **Chi** (χ): $A[x, y, z] \leftarrow A[x, y, z] \oplus ((A[(x+1), y, z] \oplus 1) \cdot A[(x+2), y, z])$ the dot is equivalent to the Boolean "AND" operation.
- 8: **Iota** (ι): $A[0, 0, z] \leftarrow A[0, 0, z] \oplus RC[i_r]$
 i_r varies from 0 to $n_r - 1$

III. HARDWARE ARCHITECTURE

This section presents the hardware architecture of the ASCON-KECCACK AEAD algorithm.

A. FPGA Implementation

The architecture of the ASCON-KECCAK as shown in Fig. 3 is structured into three primary components: the Controller, the Encryption and Decryption Unit, and the KECCAK Permutation Engine. The controller governs the overall operation, managing clock synchronization, reset control, encryption/decryption mode selection, and state transitions. It employs a finite state machine (FSM) to orchestrate different phases, including initialization, processing of associated data, encryption/decryption, and finalization. The controller ensures

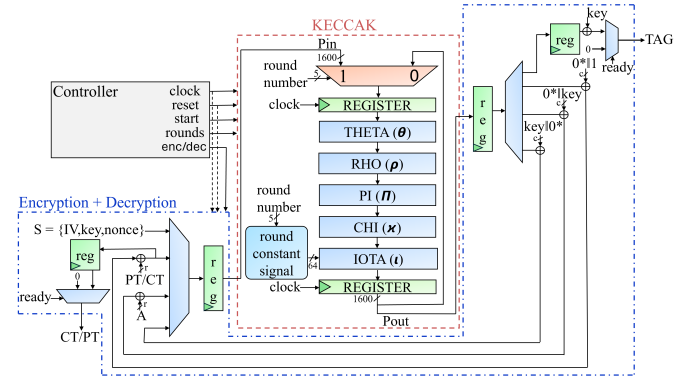


Fig. 3. ASCON-KECCAK AEAD Hardware Architecture

seamless execution by controlling the number of rounds and coordinating data flow between the internal components. The encryption and decryption unit handles the core cryptographic operations, processing plaintext, ciphertext, associated data, key, and nonce. During encryption, the plaintext is exored with the KECCAK state to generate the ciphertext, while decryption reverses this operation to retrieve the original plaintext. The unit ensures data integrity by incorporating associated data into the authentication process, and the final tag is derived after the last permutation step.

B. KECCAK

The KECCAK-p[1600,24] [8] acts as the fundamental processing block, applying sponge based transformations across multiple rounds to update the 1600-bit internal state. The ASCON algorithm utilizes a 320-bit internal state, whereas replacing it with KECCAK sponge expands the state size to 1600 bits and increases the number of permutation rounds to 24, thereby enhancing diffusion and security strength. The block consists of sequential stages: *Theta* (θ), *Rho* (ρ), *Pi* (π),

Chi (χ) and *Iota* (ι), each playing a vital role in enhancing diffusion and nonlinearity. The naming convention where a row has 5 bits (y, z constant), a column has 5 bits (x, z constant), and a lane has 64 bits (x, y constant) aids in describing KECCAK. The process begins with the *Pin*, which serves as the initial input stored in a register bank. *Theta* layer performs column-wise *XOR* operations to mix the state, followed by *Rho*, applies translations within lanes to enhance dispersion across sets of 25 bits with a constant z coordinate. The stage *Pi* rearranges the matrix to ensure further mixing of data. Whereas stage, *Chi* introduces non-linear mixing through bitwise *AND* and *XOR* operations, while *Iota* incorporates the round constants into the state to ensure cryptographic diversity, influenced by the current round number. The transformed output, *Pout*, represents the updated state after the final stage.

C. Rho Layer

The *Rho* (ρ) transformation rotates the bits within each lane by a specific offset, determined by the fixed coordinates (x, y) of the lane. This transformation modifies the z -coordinate of each bit in the lane by adding the corresponding offset, while ensuring it remains within the lane size through modular arithmetic. Instead of using modular arithmetic with the lane size (64) to keep the result within bounds, our design employs an *AND* operation with a 6-bit binary mask ("111111"), which effectively achieves the same functionality by restricting the result to a 6-bit range, thereby enhancing computational efficiency in hardware implementation. The correctness of this approach is validated in Table III, where identical results from both methods confirm that a bitwise *AND* with a binary mask can effectively replace modular arithmetic, eliminating the need for division and modulo operations.

TABLE III
MODULAR ARITHMETIC AND BITWISE AND MASKING

Input Value	Modular Arithmetic	Bitwise AND
70	$70 \bmod 64 = 6$	$70 \& 0x3F = 6$
127	$127 \bmod 64 = 63$	$127 \& 0x3F = 63$
192	$192 \bmod 64 = 0$	$127 \& 0x3F = 0$
205	$205 \bmod 64 = 13$	$127 \& 0x3F = 13$
319	$319 \bmod 64 = 63$	$127 \& 0x3F = 63$

IV. RESULTS AND DISCUSSIONS

The proposed designs are implemented using Verilog HDL, and Xilinx Vivado 2022 is utilized to deploy ASCON-KECCAK on Kintex-7 and Artix-7 FPGAs for both encryption and decryption processes. Encrypted data is transmitted from the FPGA to a server via UART, and the same interface is used to retrieve the data for decryption using the corresponding decryption key. Fig. 4 illustrates the experimental setup for the ASCON-KECCAK²⁵⁶ FPGA implementation, demonstrating the encryption and decryption of a test image using the proposed ASCON-KECCAK²⁵⁶ design. The proposed designs are validated using performance metrics such as throughput, which is defined as the number of bits processed within a given time, calculated as $\frac{\text{Number of bits processed} \times \text{Frequency}}{\text{Clock cycles}}$. In

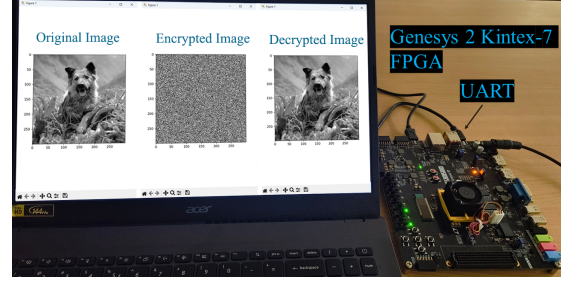


Fig. 4. Image encryption and decryption on FPGA

TABLE IV
FPGA-IMPLEMENTATION OF PROPOSED DESIGNS

Algorithm	Frequency (MHz)	LUT	FF	Throughput (Gbps)	TP/LUT (Mbps/LUT)
Kintex-7 Implementations					
KECCAK-p[1600,24]	313.47	2970	1607	20	6.75
ASCON-KECCAK ²⁵⁶	217.53	6523	4939	3.32	0.51
ASCON-KECCAK ⁵¹²	217.20	7126	4980	3.32	0.47
Artix-7 Implementations					
KECCAK-p[1600,24]	244.31	2970	1607	15.63	5.26
ASCON-KECCAK ²⁵⁶	174.76	6523	4939	2.67	0.41
ASCON-KECCAK ⁵¹²	174.76	7210	4939	2.67	0.37

[20], various interleaving techniques were implemented in the KECCAK-p module to enhance the performance of SHA-3, with the designs configured to generate a 512-bit hash output. The design achieved a throughput of 11.43 Gbps, while the highest throughput of 24.43 Gbps was reported for the optimized cascade-p architecture on a Virtex-7 for SHA-3.

A. FPGA Benchmarks

Table IV illustrates the FPGA implementation results of ASCON-KECCAK and KECCAK-p[1600,24] across Kintex-7 and Artix-7 platforms, highlighting their performance in terms of operating frequency, resource utilization, throughput and throughput per LUT (TP/LUT). The implementation of ASCON-KECCAK on Kintex-7 and Artix-7 FPGA platforms demonstrates its efficiency in achieving high throughput while maintaining a balanced resource footprint. On Kintex-7, ASCON-KECCAK²⁵⁶ and ASCON-KECCAK⁵¹² operate at 217 MHz, utilizing 6523 and 7126 LUTs, respectively, while achieving a throughput of 3.32 Gbps. The implementation of KECCAK-p[1600,24] permutation reaches a frequency of 313.47 MHz, delivering a significantly higher throughput of 20 Gbps while requiring 2970 LUTs. For the Artix-7 platform, ASCON-KECCAK operate at 174 MHz, achieving a throughput of 2.67 Gbps. On the same platform, KECCAK-p[1600,24] operates at 244.31 MHz with a throughput of 15 Gbps.

As per our knowledge, this represents the first hardware implementation of ASCON-KECCAK, with no prior research providing direct comparative results. Table V presents the performance results of the C implementation for both KECCAK and ASCON sponge functions, as reported in [19]. The table demonstrates that ASCON-KECCAK is consistently faster on 64-bit architectures, where the data processing rate plays a crucial role. This rate, which determines the factor at which data is processed, is significantly higher compared to ASCON AEAD [6]. ASCON-KECCAK enhances security by integrating KECCAK-p[1600,24], which improves diffusion

TABLE V
ENCRYPTION/DECRYPTION OF 1 GB DATA WITH ASCON AND
ASCON-KECCAK[19]

Hardware	Word Size (bits)	ASCON-128	ASCON-128a	ASCON-KECCAK ²⁵⁶	ASCON-KECCAK ⁵¹²
AMD Ryzen 9 5950X	64-bit	11.33s	9.86s	7.91s	12.99s
ARM Cortex-A72	64-bit	32.93s	23.08s	19.54s	31.96s
ARM Cortex-A76	64-bit	18.23s	13.03s	10.12s	16.81s

and nonlinearity. By leveraging KECCAK's step mappings, it offers stronger resistance against key recovery, positioning ASCON-KECCAK as a more robust cryptographic solution. For data confidentiality, ASCON-KECCAK²⁵⁶ and ASCON-KECCAK⁵¹² ensure 256-bit and 512-bit classical security, respectively, when the maximum tag size is selected [19]. These results establish a valuable reference for future studies while highlighting ASCON-KECCAK's potential in cryptographic applications requiring efficiency and high-speed processing.

B. ASIC Results

The ASIC benchmarks for the proposed designs using UMC 90 nm technology are summarized in Table VI. KECCAK-p[1600,24] performs 24 rounds of permutation on a 1600-bit state. Both ASCON-KECCAK²⁵⁶ and ASCON-KECCAK⁵¹² share a common permutation core for encryption, decryption, and generating a 128-bit tag. ASCON-KECCAK²⁵⁶ operates with a 256-bit key, processing 512-bit plaintext, ciphertext, and associated data, while ASCON-KECCAK⁵¹² uses a 512-bit key for the same data dimensions. ASIC synthesis was performed for a clock period of 200 ps (equivalent to 5000 MHz) and additional details are reported in Table VI.

TABLE VI
AREA BENCHMARKS FOR ASIC (UMC 90 nm)

Design	Instances	Area (μm^2)	Timing slack (ps)	Total Power (mW)
KECCAK-p[1600,24]	13850	115843	563	46.86
ASCON-KECCAK ²⁵⁶	30913	244980	7	74.2
ASCON-KECCAK ⁵¹²	26345	221382	1	66.74

V. CONCLUSION

This work presents a novel hardware implementation of the ASCON-KECCAK AEAD algorithm, an efficient integration of ASCON's cryptographic framework with the KECCAK permutation, achieving a balance between security, performance, and hardware utilization. FPGA results on Kintex-7 show a throughput of 3.32 Gbps for ASCON-KECCAK and 20 Gbps for KECCAK-p[1600,24], with resource utilization of 6.5k LUTs for ASCON-KECCAK²⁵⁶, 7.2k LUTs for ASCON-KECCAK⁵¹², and 2.9k LUTs for KECCAK-p[1600,24]. ASIC benchmarking further supports its efficiency. It emerges as a promising candidate for secure encryption, setting a benchmark for future cryptographic solutions.

REFERENCES

- [1] M. Frustaci, P. Pace, G. Aloï, and G. Fortino, "Evaluating critical security issues of the iot world: Present and future challenges," *IEEE Internet of Things Journal*, vol. 5, no. 4, pp. 2483–2495, 2018.
- [2] T. Dorigo, G. D. Brown, C. Casonato, A. Cerdà, J. Ciarrochi, M. da Lio, N. D'Souza, N. R. Gauger, S. C. Hayes, S. G. Hofmann, R. Johansson, M. Liwicki, F. Lotte, J. J. Nieto, G. Olivato, P. Parnes, G. Perry, A. Plebe, I. M. Rao, N. Rezaei, F. Sandin, A. Ustyuzhanin, G. Vallortigara, P. Vischia, and N. Yazdanpanah, "Artificial intelligence in science and society: The vision of users," *IEEE Access*, vol. 13, pp. 15 993–16 054, 2025.
- [3] H.-N. Dai, Z. Zheng, and Y. Zhang, "Blockchain for internet of things: A survey," *IEEE Internet of Things Journal*, vol. 6, no. 5, pp. 8076–8094, 2019.
- [4] L. Chettri and R. Bera, "A comprehensive survey on internet of things (iot) toward 5g wireless systems," *IEEE Internet of Things Journal*, vol. 7, no. 1, pp. 16–32, 2020.
- [5] E. Ismagilova, L. Hughes, N. P. Rana, and Y. K. Dwivedi, "Security, privacy and risks within smart cities: Literature review and development of a smart city interaction framework," *Information Systems Frontiers*, vol. 24, no. 2, pp. 393–414, 2022.
- [6] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schl  ffer, "ASCON specification," <https://ascon.iaink.tugraz.at/specification.html>, Accessed May 2023, 2021.
- [7] P. Rogaway, "Authenticated-encryption with associated-data," in *Proceedings of the 9th ACM Conference on Computer and Communications Security*, ser. CCS '02. New York, NY, USA: Association for Computing Machinery, 2002, p. 98–107. [Online]. Available: <https://doi.org/10.1145/586110.586125>
- [8] N. I. of Standards, T. (NIST), and M. J. Dworkin, "SHA-3 standard: Permutation-based hash and extendable-output functions," 2015-08-04 00:08:00 2015. [Online]. Available: https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=919061
- [9] J. Kelsey, S. jen Chang, and R. Perlner, "SHA-3 derived functions: cshake, kmac, tuplehash and parallelhash," 2016-12-22 00:12:00 2016. [Online]. Available: https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=922422
- [10] Y. Yang, D. He, N. Kumar, and S. Zeadally, "Compact hardware implementation of a sha-3 core for wireless body sensor networks," *IEEE Access*, vol. 6, pp. 40 128–40 136, 2018.
- [11] J.-P. Aumasson, L. Henzen, W. Meier, and R. C.-W. Phan, "BLAKE: A SHA-3 Candidate," National Institute of Standards and Technology (NIST), Gaithersburg, MD, USA, Tech. Rep., 2008. [Online]. Available: https://perso.uclouvain.be/fstandae/source_codes/hash_atmel/specs/blake.pdf
- [12] H. Wu, "The hash function JH," Nanyang Technological University, Tech. Rep., 2011, round 3 submission to NIST SHA-3 competition. [Online]. Available: https://www3.ntu.edu.sg/home/wuhj/research/jh/jh_round3.pdf
- [13] S. jen Chang, R. Perlner, W. E. Burr, M. S. Turan, J. M. Kelsey, S. Paul, and L. E. Bassham, "Third-round report of the SHA-3 cryptographic hash algorithm competition," National Institute of Standards and Technology (NIST), Tech. Rep. NIST IR 7896, 2012. [Online]. Available: <https://doi.org/10.6028/NIST.IR.7896>
- [14] National Institute of Standards and Technology (NIST), "Advanced encryption standard (AES)," U.S. Department of Commerce, Gaithersburg, MD, USA, Tech. Rep. FIPS-197, Nov. 2001. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf>
- [15] D.-e.-S. Kundi, A. Khalid, A. Aziz, C. Wang, M. O'Neill, and W. Liu, "Resource-shared crypto-coprocessor of aes enc/dec with sha-3," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 67, no. 12, pp. 4869–4882, 2020.
- [16] S. Bayat-Sarmadi, M. Mozaffari-Kermani, and A. Reyhani-Masoleh, "Efficient and concurrent reliable realization of the secure cryptographic SHA-3 algorithm," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 33, no. 7, pp. 1105–1109, 2014.
- [17] J. Kaur, M. Mozaffari Kermani, and R. Azarderakhsh, "Hardware constructions for error detection in lightweight authenticated cipher ASCON benchmarked on FPGA," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 69, no. 4, pp. 2276–2280, 2022.
- [18] K.-D. Nguyen, T.-K. Dang, B. Kieu-Do-Nguyen, D.-H. Le, C.-K. Pham, and T.-T. Hoang, "ASIC implementation of ASCON lightweight cryptography for IoT applications," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 72, no. 1, pp. 278–282, 2025.
- [19] S. M  ller, "Ascon-keccak AEAD algorithm," Cryptology ePrint Archive, Paper 2024/858, 2024. [Online]. Available: <https://eprint.iacr.org/2024/858>
- [20] M. M. Sravani and S. A. Durai, "On efficiency enhancement of SHA-3 for FPGA-based multimodal biometric authentication," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 30, no. 4, pp. 488–501, 2022.